Single Inheritance

In single inheritance, a single subclass extends from a single superclass. This means that one class inherits the properties and methods of another class.

Example of Single Inheritance

```
java
VerifyOpen In EditorRunCopy code
1// Superclass
2class Animal {
3    protected String name;
4
5    public void display() {
6        System.out.println("I am an animal.");
7    }
8}
9
10// Subclass
11class Dog extends Animal {
12    public void setName(String name) {
13        this.name = name;
14    }
15
16    public void displayDog() {
17        System.out.println("My name is " + name);
18    }
19}
20
21// Main class to test the inheritance
22public class Main {
23    public static void main(String[] args) {
24        Dog labrador = new Dog();
25        labrador.setName("Rocky");
26        labrador.display();      // Calls method from Animal class
27        labrador.displayDog();   // Calls method from Dog class
28    }
29}
```

Output:

```
VerifyOpen In EditorRunCopy code
1I am an animal.
2My name is Rocky
```

Multilevel Inheritance

In multilevel inheritance, a subclass extends from a superclass, and then that subclass can act as a superclass for another subclass. This creates a chain of inheritance.

Example of Multilevel Inheritance

```
java
VerifyOpen In EditorRunCopy code
1// Superclass
2class Animal {
3    public void display() {
4        System.out.println("I am an animal.");
5    }
6}
7
```

```
8// Intermediate subclass
9class Dog extends Animal {
10    public void bark() {
11        System.out.println("Woof! Woof!");
12    }
13}
14
15// Subclass of Dog
16class Labrador extends Dog {
17    public void displayBreed() {
18        System.out.println("I am a Labrador.");
19    }
20}
21
22// Main class to test the inheritance
23public class Main {
24    public static void main(String[] args) {
25        Labrador labrador = new Labrador();
26        labrador.display();      // Calls method from Animal class
27        labrador.bark();        // Calls method from Dog class
28        labrador.displayBreed(); // Calls method from Labrador class
29    }
30}
```

Output:

```
VerifyOpen In EditorRunCopy code
1I am an animal.
2Woof! Woof!
3I am a Labrador.
```

Key Points

- **Single Inheritance**: One subclass inherits from one superclass. This promotes code reusability and method overriding.
- **Multilevel Inheritance**: A subclass can inherit from another subclass, creating a hierarchy of classes.
- **Java Limitations**: Java does not support multiple inheritance with classes to avoid complexity and ambiguity. However, it can be achieved using interfaces.

These examples illustrate how inheritance works in Java, allowing for organized and reusable code structures. If you have any further questions or need more examples, feel free to ask!

Hierarchical Inheritance in Java

Definition: Hierarchical inheritance in Java occurs when multiple subclasses inherit from a single superclass. This allows the subclasses to reuse the properties and methods of the parent class, promoting code reusability and modularity.

Key Features of Hierarchical Inheritance:

- **Single Parent Class:** There is one parent class (superclass) from which multiple child classes (subclasses) inherit.
- **Code Reusability:** Subclasses can use the methods and properties of the parent class without redefining them.

- **IS-A Relationship:** Each subclass represents a specialized version of the parent class.

Example of Hierarchical Inheritance

Here's a simple example demonstrating hierarchical inheritance in Java:

```
java
VerifyOpen In EditorRunCopy code
1// Parent class
2class Animal {
3    void eat() {
4        System.out.println("eating...");
5    }
6}
7
8// First child class
9class Dog extends Animal {
10    void bark() {
11        System.out.println("barking...");
12    }
13}
14
15// Second child class
16class Cat extends Animal {
17    void meow() {
18        System.out.println("meowing...");
19    }
20}
21
22// Test class to demonstrate hierarchical inheritance
23public class TestInheritance {
24    public static void main(String[] args) {
25        Dog dog = new Dog();
26        dog.bark(); // Dog specific method
27        dog.eat();  // Inherited method from Animal
28
29        Cat cat = new Cat();
30        cat.meow(); // Cat specific method
31        cat.eat();  // Inherited method from Animal
32    }
33}
```

Output:

```
VerifyOpen In EditorRunCopy code
1barking...
2eating...
3meowing...
4eating...
```

Explanation:

- **Animal Class:** This is the parent class with a method `eat()`.
- **Dog Class:** This subclass extends `Animal` and adds a method `bark()`.
- **Cat Class:** This subclass also extends `Animal` and adds a method `meow()`.
- In the `main` method, we create instances of `Dog` and `Cat`, demonstrating that both subclasses can call the inherited `eat()` method from the `Animal` class.

Benefits of Hierarchical Inheritance:

- **Reduced Code Duplication:** Common functionality is defined in the parent class, reducing redundancy.
- **Easier Maintenance:** Changes in the parent class automatically propagate to all subclasses.
- **Clear Structure:** It creates a clear and organized class hierarchy, making the code easier to understand.

Conclusion

Hierarchical inheritance is a powerful feature in Java that allows for the creation of a well-structured class hierarchy, promoting code reuse and maintainability. By understanding and implementing hierarchical inheritance, developers can create more efficient and organized codebases.

In Java, **association**, **aggregation**, and **composition** are fundamental concepts that describe relationships between classes. Understanding these relationships is crucial for designing robust object-oriented systems. Below, I will explain each concept along with examples.

1. Association

- **Definition**: Association is a general term that describes a relationship between two independent classes. It can be one-to-one, one-to-many, many-to-one, or many-to-many.
- **Characteristics**:
    - Classes can exist independently.
    - No ownership is implied.

Example of Association

```
java
VerifyOpen In EditorRunCopy code
1// Java Program to illustrate the Concept of Association
2
3import java.util.HashSet;
4import java.util.Set;
5
6// Class 1: Bank
7class Bank {
8    private String bankName;
9    private Set<Employee> employees;
10
11    public Bank(String bankName) {
12        this.bankName = bankName;
13        this.employees = new HashSet<>();
14    }
15
16    public void addEmployee(Employee employee) {
17        employees.add(employee);
18    }
19
20    public String getBankName() {
21        return bankName;
22    }
23
24    public Set<Employee> getEmployees() {
```

```
25        return employees;
26    }
27}
28
29// Class 2: Employee
30class Employee {
31    private String name;
32
33    public Employee(String name) {
34        this.name = name;
35    }
36
37    public String getName() {
38        return name;
39    }
40}
41
42// Class 3: Main class
43public class AssociationExample {
44    public static void main(String[] args) {
45        Bank bank = new Bank("ICICI");
46        Employee emp1 = new Employee("Ridhi");
47        Employee emp2 = new Employee("Vijay");
48
49        bank.addEmployee(emp1);
50        bank.addEmployee(emp2);
51
52        for (Employee emp : bank.getEmployees()) {
53            System.out.println(emp.getName() + " belongs to bank " +
bank.getBankName());
54        }
55    }
56}
```

2. Aggregation

- **Definition**: Aggregation is a special form of association that represents a "has-a" relationship. It implies a relationship where one class (the whole) contains a collection of other classes (the parts), but the parts can exist independently of the whole.
- **Characteristics**:
    - The contained objects can exist independently of the container object.
    - It is a unidirectional association.

Example of Aggregation

```
java
VerifyOpen In EditorRunCopy code
1// Java Program to illustrate the Concept of Aggregation
2
3import java.util.ArrayList;
4import java.util.List;
5
6// Class 1: Student
7class Student {
8    private String studentName;
9    private int studentId;
10
11    public Student(String studentName, int studentId) {
12        this.studentName = studentName;
13        this.studentId = studentId;
14    }
```

```
15
16    public String getStudentName() {
17        return studentName;
18    }
19}
20
21// Class 2: Department
22class Department {
23    private String deptName;
24    private List<Student> students;
25
26    public Department(String deptName) {
27        this.deptName = deptName;
28        this.students = new ArrayList<>();
29    }
30
31    public void addStudent(Student student) {
32        students.add(student);
33    }
34
35    public List<Student> getStudents() {
36        return students;
37    }
38}
39
40// Class 3: Institute
41class Institute {
42    private String instituteName;
43    private List<Department> departments;
44
45    public Institute(String instituteName) {
46        this.instituteName = instituteName;
47        this.departments = new ArrayList<>();
48    }
49
50    public void addDepartment(Department department) {
51        departments.add(department);
52    }
53
54    public int getTotalStudents() {
55        int total = 0;
56        for (Department dept : departments) {
57            total += dept.getStudents().size();
58        }
59        return total;
60    }
61}
62
63// Class 4: Main class
64public class AggregationExample {
65    public static void main(String[] args) {
66        Student s1 = new Student("Parul", 1);
67        Student s2 = new Student("Sachin", 2);
68        Department cse = new Department("CSE");
69        cse.addStudent(s1);
70        cse.addStudent(s2);
71
72        Institute institute = new Institute("BITS");
73        institute.addDepartment(cse);
74
75        System.out.println("Total students in institute: " +
institute.getTotalStudents());
76    }
77}
```

3. Composition

- **Definition**: Composition is a stronger form of aggregation that implies a "part-of" relationship. In this relationship, the contained objects cannot exist independently of the container object. If the container is destroyed, so are the contained objects.
- **Characteristics**:
    - The contained objects are dependent on the container object.
    - It represents a strong ownership relationship.

Example of Composition

```
java
VerifyOpen In EditorRunCopy code
1// Java Program to illustrate the Concept of Composition
2
3// Class 1: Engine
4class Engine {
5    private String type;
6
7    public Engine(String type) {
8        this.type = type;
9    }
10
11    public String getType() {
12        return type;
13    }
14}
15
16// Class 2: Car
17class Car {
18    private Engine engine;
19
20    public Car(String engineType) {
21        this.engine = new Engine(engineType); // Engine is created with Car
22    }
23
24    public Engine getEngine() {
25        return engine;
26    }
27}
28
29// Class 3: Main class
30public class CompositionExample {
31    public static void main(String[] args) {
32        Car car = new Car("V8");
33        System.out.println("Car engine type: " + car.getEngine().getType());
34    }
35}
```

Summary of Differences

Feature	Association	Aggregation	Composition
Definition	General relationship	Has-a relationship	Part-of relationship
Dependency	Independent lifecycles	Independent lifecycles	Dependent lifecycles
Ownership	No ownership implied	Shared ownership	Exclusive ownership
Strength	Weak	Moderate	Strong
Example	Teacher and Student	Library and Books	Car and Engine

Understanding these concepts helps in designing better object-oriented systems by clearly defining how different classes interact with each other.

Compile-time vs Runtime Polymorphism in Java

Polymorphism is a core concept in Object-Oriented Programming (OOP) that allows methods to do different things based on the object that it is acting upon. In Java, polymorphism can be categorized into two types: **Compile-time polymorphism** and **Runtime polymorphism**.

---

1. Compile-time Polymorphism

- **Definition**: Compile-time polymorphism, also known as static polymorphism or early binding, occurs when the method to be executed is determined at compile time. This is typically achieved through **method overloading**.
- **How it Works**: The compiler determines which method to call based on the method signature (method name and parameter types).
- **Example**: Below is an example demonstrating method overloading.

```
java
VerifyOpen In EditorRunCopy code
1// Java program to demonstrate compile-time polymorphism
2public class CompileTimePolymorphism {
3
4    // Method to add two integers
5    public static int add(int a, int b) {
6        return a + b;
7    }
8
9    // Method to add three integers
10    public static int add(int a, int b, int c) {
11        return a + b + c;
12    }
13
14    // Method to add two doubles
15    public static double add(double a, double b) {
16        return a + b;
17    }
18
19    // Driver code
20    public static void main(String[] args) {
21        System.out.println("Sum of two integers: " + add(2, 3)); // Calls
add(int, int)
22        System.out.println("Sum of three integers: " + add(2, 3, 4)); // Calls
add(int, int, int)
23        System.out.println("Sum of two doubles: " + add(2.0, 3.0)); // Calls
add(double, double)
24    }
25}
```

Output:

```
VerifyOpen In EditorRunCopy code
1Sum of two integers: 5
2Sum of three integers: 9
3Sum of two doubles: 5.0
```

---

2. Runtime Polymorphism

- **Definition**: Runtime polymorphism, also known as dynamic polymorphism or late binding, occurs when the method to be executed is determined at runtime. This is achieved through **method overriding**.
- **How it Works**: The Java Virtual Machine (JVM) determines which method to call based on the object type at runtime, not the reference type.
- **Example**: Below is an example demonstrating method overriding.

```
java
VerifyOpen In EditorRunCopy code
1// Base class
2class Animal {
3    void sound() {
4        System.out.println("Animal makes a sound");
5    }
6}
7
8// Derived class
9class Dog extends Animal {
10    @Override
11    void sound() {
12        System.out.println("Dog barks");
13    }
14}
15
16// Another derived class
17class Cat extends Animal {
18    @Override
19    void sound() {
20        System.out.println("Cat meows");
21    }
22}
23
24// Driver code
25public class RuntimePolymorphism {
26    public static void main(String[] args) {
27        Animal myDog = new Dog(); // Animal reference but Dog object
28        Animal myCat = new Cat(); // Animal reference but Cat object
29
30        myDog.sound(); // Calls Dog's sound method
31        myCat.sound(); // Calls Cat's sound method
32    }
33}
```

Output:

```
VerifyOpen In EditorRunCopy code
1Dog barks
2Cat meows
```

Key Differences Between Compile-time and Runtime Polymorphism

Aspect	Compile-time Polymorphism	Runtime Polymorphism
Binding Time	Resolved at compile time (static binding)	Resolved at runtime (dynamic binding)
Implementation	Achieved through method overloading	Achieved through method overriding
Flexibility	Less flexible, as all decisions are made at compile time	More flexible, as decisions are made at runtime
Inheritance	Not required	Required (involves inheritance)
Performance	Faster execution	Slower execution due to dynamic method resolution

---

By understanding these two types of polymorphism, Java developers can write more flexible and maintainable code. If you have any further questions or need more examples, feel free to ask!

Certainly! Let's explore the **rules of method overriding and overloading in Java**, along with examples to illustrate each concept.

Method Overloading

Definition: Method overloading occurs when two or more methods in the same class have the same name but different parameter lists (different type or number of parameters). It is a form of compile-time polymorphism.

Rules for Method Overloading

- **Same Method Name**: All overloaded methods must have the same name.
- **Different Parameters**: The parameter list must differ in type, number, or both.
- **Return Type**: The return type can be different, but it does not contribute to overloading.
- **Access Modifiers**: Can be different.
- **Static Methods**: Can also be overloaded.

Example of Method Overloading

```
java
VerifyOpen In EditorRunCopy code
1public class Calculator {
2    // Overloaded method for adding two integers
3    public int add(int a, int b) {
4        return a + b;
5    }
6
7    // Overloaded method for adding three integers
8    public int add(int a, int b, int c) {
9        return a + b + c;
10    }
11
12    // Overloaded method for adding two doubles
13    public double add(double a, double b) {
14        return a + b;
15    }
16
```

```
17    public static void main(String[] args) {
18        Calculator calc = new Calculator();
19        System.out.println("Sum of 2 and 3: " + calc.add(2, 3)); // Calls
add(int, int)
20        System.out.println("Sum of 2, 3, and 4: " + calc.add(2, 3, 4)); //
Calls add(int, int, int)
21        System.out.println("Sum of 2.5 and 3.5: " + calc.add(2.5, 3.5)); //
Calls add(double, double)
22    }
23}
```

Method Overriding

Definition: Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its superclass. It is a form of runtime polymorphism.

Rules for Method Overriding

- **Same Method Name**: The method in the subclass must have the same name as the method in the superclass.
- **Same Parameters**: The parameter list must be identical (same type and number).
- **Return Type**: The return type must be the same or a subtype (covariant return type).
- **Access Modifiers**: The access level can be the same or more accessible (e.g., `protected` in superclass can be `public` in subclass).
- **Cannot Override**: Static methods, final methods, and private methods cannot be overridden.

Example of Method Overriding

```
java
VerifyOpen In EditorRunCopy code
1class Animal {
2    void sound() {
3        System.out.println("Animal makes a sound");
4    }
5}
6
7class Dog extends Animal {
8    @Override
9    void sound() {
10        System.out.println("Dog barks");
11    }
12}
13
14public class Main {
15    public static void main(String[] args) {
16        Animal myAnimal = new Animal(); // Animal reference and object
17        Animal myDog = new Dog(); // Animal reference but Dog object
18
19        myAnimal.sound(); // Outputs: Animal makes a sound
20        myDog.sound(); // Outputs: Dog barks (overridden method)
21    }
22}
```

Key Differences Between Overloading and Overriding

Aspect	Method Overloading	Method Overriding
Definition	Same method name, different parameters	Same method name and parameters in subclass
Polymorphism Type	Compile-time polymorphism	Runtime polymorphism
Class Relationship	Within the same class	Between superclass and subclass
Return Type	Can be different	Must be the same or a subtype
Access Modifiers	Can be different	Must be the same or more accessible
Static Methods	Can be overloaded	Cannot be overridden

Understanding these concepts is crucial for effective Java programming, especially when working with inheritance and polymorphism. If you have any further questions or need more examples, feel free to ask!

Certainly! Let's explore the **`super`** and **`this`** keywords in Java, along with their uses and examples.

Overview of `this` and `super`

- **`this` Keyword**:
  - Refers to the current instance of the class.
  - Used to access instance variables and methods of the current class.
  - Can be used to call another constructor in the same class (constructor chaining).
- **`super` Keyword**:
  - Refers to the immediate parent class instance.
  - Used to access parent class methods and constructors.
  - Can be used to call the parent class constructor from a child class.

Key Differences

Feature	`this`	`super`
Refers to	Current class instance	Immediate parent class instance
Constructor call	Calls another constructor in the same class	Calls a constructor in the parent class
Access	Current class methods and variables	Parent class methods and variables
Usage in static context	Can be used in instance methods only	Cannot be used in static context

Examples

Example of `this` Keyword

```
java
VerifyOpen In EditorRunCopy code
1class Example {
2    private int value;
3
4    // Constructor
5    public Example(int value) {
```

```
6        this.value = value; // 'this.value' refers to the instance variable
7    }
8
9    public void display() {
10        System.out.println("Value: " + this.value); // Accessing instance
variable
11    }
12}
13
14public class TestThis {
15    public static void main(String[] args) {
16        Example obj = new Example(10);
17        obj.display(); // Output: Value: 10
18    }
19}
```

Example of `super` Keyword

```
java
VerifyOpen In EditorRunCopy code
1class Parent {
2    public Parent() {
3        System.out.println("Parent Constructor");
4    }
5
6    public void display() {
7        System.out.println("Display from Parent");
8    }
9}
10
11class Child extends Parent {
12    public Child() {
13        super(); // Calls Parent constructor
14        System.out.println("Child Constructor");
15    }
16
17    public void display() {
18        super.display(); // Calls Parent's display method
19        System.out.println("Display from Child");
20    }
21}
22
23public class TestSuper {
24    public static void main(String[] args) {
25        Child obj = new Child(); // Output: Parent Constructor, Child
Constructor
26        obj.display(); // Output: Display from Parent, Display from Child
27    }
28}
```

Constructor Chaining with `this` and `super`

- **`this()`** can be used to call another constructor in the same class.
- **`super()`** must be the first statement in a child class constructor to call the parent class constructor.

Example of Constructor Chaining

```
java
VerifyOpen In EditorRunCopy code
1class Base {
```

```
2    Base() {
3        System.out.println("Base Constructor");
4    }
5}
6
7class Derived extends Base {
8    Derived() {
9        super(); // Calls Base constructor
10        System.out.println("Derived Constructor");
11    }
12}
13
14class AnotherDerived extends Derived {
15    AnotherDerived() {
16        this(10); // Calls AnotherDerived(int) constructor
17        System.out.println("AnotherDerived Constructor");
18    }
19
20    AnotherDerived(int x) {
21        super(); // Calls Derived constructor
22        System.out.println("AnotherDerived with value: " + x);
23    }
24}
25
26public class TestChaining {
27    public static void main(String[] args) {
28        AnotherDerived obj = new AnotherDerived();
29        // Output: Base Constructor, Derived Constructor, AnotherDerived with
value: 10, AnotherDerived Constructor
30    }
31}
```

Important Points to Remember

- **`this()`** and **`super()`** cannot be used together in the same constructor.
- Both must be the first statement if used.
- You cannot call `this()` or `super()` recursively.

Conclusion

The **`this`** and **`super`** keywords are essential for managing class relationships and constructor behavior in Java. Understanding their usage helps in writing cleaner and more efficient object-oriented code. If you have any further questions or need more examples, feel free to ask!